

Lesson 4: Pandas Advanced - Data Manipulation

Xử lý dữ liệu nâng cao với Pandas

Lesson Information | Thông tin bài học

Item	Description
Duration	90 minutes
Type	Lecture + Lab
Prerequisites	Lesson 3: Pandas Basics

Learning Objectives | Mục tiêu bài học

#	Objective	Assessment
1	Handle missing data effectively	Code exercises
2	Transform and reshape data	Lab work
3	Use GroupBy for aggregation	Code exercises
4	Merge and join DataFrames	Quiz

1. Handling Missing Data (20 min)

1.1 Detecting Missing Values

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'A': [1, 2, np.nan, 4],
    'B': [5, np.nan, np.nan, 8],
    'C': ['x', 'y', 'z', None]
})

# Check for missing
print(df.isnull())          # True where NaN
print(df.notnull())         # True where not NaN
print(df.isna())           # Same as isnull

# Count missing values
print(df.isnull().sum())    # Per column
print(df.isnull().sum().sum()) # Total
```

```
# Percentage missing
print(df.isnull().mean() * 100)
```

1.2 Handling Missing Values

```
# Drop rows with any NaN
df_dropped = df.dropna()

# Drop rows where all values are NaN
df_dropped = df.dropna(how='all')

# Drop rows with NaN in specific columns
df_dropped = df.dropna(subset=['A', 'B'])

# Drop columns with NaN
df_dropped = df.dropna(axis=1)

# Threshold: keep rows with at least n non-NaN
df_dropped = df.dropna(thresh=2)
```

1.3 Filling Missing Values

```
# Fill with specific value
df_filled = df.fillna(0)

# Fill with different values per column
df_filled = df.fillna({'A': 0, 'B': df['B'].mean(), 'C': 'unknown'})

# Forward fill (use previous value)
df_filled = df.fillna(method='ffill')

# Backward fill (use next value)
df_filled = df.fillna(method='bfill')

# Fill with statistics
df['A'] = df['A'].fillna(df['A'].mean())
df['A'] = df['A'].fillna(df['A'].median())
df['C'] = df['C'].fillna(df['C'].mode()[0])

# Interpolation
df['A'] = df['A'].interpolate()
df['A'] = df['A'].interpolate(method='linear')
```

1.4 Replace Values

```
# Replace specific values
df.replace(0, np.nan)
df.replace([0, -1], np.nan)
df.replace({'A': 0, 'B': -1}, np.nan)

# Replace with regex
df['C'].replace(r'^x.*', 'X', regex=True)
```

2. Data Transformation (20 min)

2.1 Adding and Removing Columns

```
df = pd.DataFrame({
    'Name': ['An', 'Binh', 'Cuong'],
    'Math': [80, 90, 85],
    'English': [75, 88, 92]
})

# Add column
df['Total'] = df['Math'] + df['English']
df['Average'] = df[['Math', 'English']].mean(axis=1)

# Insert at specific position
df.insert(1, 'ID', [1, 2, 3])

# Using assign (returns new DataFrame)
df = df.assign(
    Total=df['Math'] + df['English'],
    Passed=lambda x: x['Total'] > 150
)

# Drop columns
df = df.drop('Total', axis=1)
df = df.drop(['Total', 'Average'], axis=1)
df.drop('Total', axis=1, inplace=True)
```

2.2 Renaming

```
# Rename columns
df = df.rename(columns={'Math': 'Mathematics', 'English':
    'English_Score'})

# Rename with function
df.columns = df.columns.str.lower()
df.columns = df.columns.str.replace(' ', '_')
```

```
# Rename index
df = df.rename(index={0: 'first', 1: 'second'})
```

2.3 Applying Functions

```
# Apply to column
df['Math_Grade'] = df['Math'].apply(lambda x: 'A' if x >= 90 else 'B' if x
>= 80 else 'C')

# Apply to DataFrame
def normalize(x):
    return (x - x.min()) / (x.max() - x.min())

df[['Math', 'English']] = df[['Math', 'English']].apply(normalize)

# Apply to each element
df['Name'] = df['Name'].apply(str.upper)

# Map values
grade_map = {'A': 4.0, 'B': 3.0, 'C': 2.0}
df['GPA'] = df['Math_Grade'].map(grade_map)

# Apply function to rows
df['Max_Score'] = df.apply(lambda row: max(row['Math'], row['English']),
axis=1)
```

2.4 Sorting

```
# Sort by column
df_sorted = df.sort_values('Math')
df_sorted = df.sort_values('Math', ascending=False)

# Sort by multiple columns
df_sorted = df.sort_values(['Math', 'English'], ascending=[False, True])

# Sort by index
df_sorted = df.sort_index()
df_sorted = df.sort_index(ascending=False)

# Reset index
df = df.reset_index(drop=True)
```

2.5 Duplicates

```
# Check duplicates
print(df.duplicated())          # Boolean Series
```

```
print(df.duplicated().sum())    # Count

# Drop duplicates
df_unique = df.drop_duplicates()
df_unique = df.drop_duplicates(subset=['Name'])
df_unique = df.drop_duplicates(keep='first') # 'first', 'last', False
```

3. GroupBy Operations (25 min)

3.1 Basic GroupBy

```
df = pd.DataFrame({
    'Department': ['Sales', 'Sales', 'HR', 'HR', 'IT', 'IT'],
    'Employee': ['An', 'Binh', 'Cuong', 'Dung', 'Em', 'Phuc'],
    'Salary': [50000, 55000, 45000, 48000, 60000, 65000],
    'Age': [25, 30, 28, 35, 27, 32]
})

# Group by single column
grouped = df.groupby('Department')

# View groups
print(grouped.groups)
print(grouped.get_group('Sales'))

# Aggregation
print(grouped['Salary'].mean())
print(grouped['Salary'].sum())
print(grouped[['Salary', 'Age']].mean())
```

3.2 Multiple Aggregations

```
# Single aggregation
print(df.groupby('Department')['Salary'].mean())

# Multiple aggregations on one column
print(df.groupby('Department')['Salary'].agg(['mean', 'min', 'max',
'std']))

# Different aggregations for different columns
print(df.groupby('Department').agg({
    'Salary': ['mean', 'sum'],
    'Age': ['min', 'max']
}))

# Named aggregations (pandas 0.25+)
print(df.groupby('Department').agg(
    avg_salary=('Salary', 'mean'),
```

```
total_salary=('Salary', 'sum'),
youngest=('Age', 'min'),
oldest=('Age', 'max'),
count=('Employee', 'count')
))
```

3.3 GroupBy with Multiple Columns

```
df = pd.DataFrame({
    'Department': ['Sales', 'Sales', 'HR', 'HR', 'Sales', 'HR'],
    'Gender': ['M', 'F', 'M', 'F', 'M', 'F'],
    'Salary': [50000, 55000, 45000, 48000, 52000, 46000]
})

# Group by multiple columns
print(df.groupby(['Department', 'Gender'])['Salary'].mean())

# Unstack to pivot
print(df.groupby(['Department', 'Gender'])['Salary'].mean().unstack())
```

3.4 Transform and Filter

```
# Transform - apply function, keep original shape
df['Salary_Normalized'] = df.groupby('Department')['Salary'].transform(
    lambda x: (x - x.mean()) / x.std()
)

# Filter groups
high_avg = df.groupby('Department').filter(
    lambda x: x['Salary'].mean() > 50000
)

# Apply custom function to each group
def top_earner(group):
    return group.nlargest(1, 'Salary')

top_by_dept = df.groupby('Department').apply(top_earner)
```

4. Merging and Joining (15 min)

4.1 Concatenation

```
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})
```

```
# Vertical concatenation (stack rows)
df_concat = pd.concat([df1, df2])
df_concat = pd.concat([df1, df2], ignore_index=True)

# Horizontal concatenation (add columns)
df_concat = pd.concat([df1, df2], axis=1)

# With keys
df_concat = pd.concat([df1, df2], keys=['first', 'second'])
```

4.2 Merge (SQL-like Join)

```
# Sample data
employees = pd.DataFrame({
    'emp_id': [1, 2, 3, 4],
    'name': ['An', 'Binh', 'Cuong', 'Dung'],
    'dept_id': [10, 20, 10, 30]
})

departments = pd.DataFrame({
    'dept_id': [10, 20, 40],
    'dept_name': ['Sales', 'HR', 'IT']
})

# Inner join (default)
merged = pd.merge(employees, departments, on='dept_id')

# Left join
merged = pd.merge(employees, departments, on='dept_id', how='left')

# Right join
merged = pd.merge(employees, departments, on='dept_id', how='right')

# Outer join
merged = pd.merge(employees, departments, on='dept_id', how='outer')

# Different column names
salaries = pd.DataFrame({
    'employee_id': [1, 2, 3],
    'salary': [50000, 60000, 55000]
})
merged = pd.merge(employees, salaries, left_on='emp_id',
    right_on='employee_id')

# Multiple keys
merged = pd.merge(df1, df2, on=['key1', 'key2'])
```

4.3 Join

```
# Join on index
df1 = pd.DataFrame({'A': [1, 2, 3]}, index=['a', 'b', 'c'])
df2 = pd.DataFrame({'B': [4, 5, 6]}, index=['a', 'b', 'd'])

joined = df1.join(df2) # Left join on index
joined = df1.join(df2, how='outer')
joined = df1.join(df2, how='inner')
```

5. Reshaping Data (10 min)

5.1 Pivot Tables

```
df = pd.DataFrame({
    'Date': ['2024-01', '2024-01', '2024-02', '2024-02'],
    'Product': ['A', 'B', 'A', 'B'],
    'Sales': [100, 150, 120, 180]
})

# Pivot
pivot = df.pivot(index='Date', columns='Product', values='Sales')
#
#      A      B
# 2024-01  100  150
# 2024-02  120  180

# Pivot table with aggregation
pivot = pd.pivot_table(df, values='Sales', index='Date',
                        columns='Product', aggfunc='sum')

# Multiple aggregations
pivot = pd.pivot_table(df, values='Sales', index='Date',
                        columns='Product', aggfunc=['sum', 'mean'])
```

5.2 Melt (Unpivot)

```
df = pd.DataFrame({
    'Name': ['An', 'Binh'],
    'Math': [80, 90],
    'English': [85, 88]
})

# Melt to long format
melted = pd.melt(df, id_vars=['Name'], value_vars=['Math', 'English'],
                 var_name='Subject', value_name='Score')
#      Name  Subject  Score
# 0     An     Math     80
# 1    Binh     Math     90
```

```
# 2    An  English    85
# 3    Binh English    88
```

5.3 Stack and Unstack

```
# Stack - pivot columns to rows
stacked = df.stack()

# Unstack - pivot rows to columns
unstacked = stacked.unstack()
```

Practice Exercises | Bài tập thực hành

Exercise 4.1: Data Cleaning

```
# Create DataFrame with:
# - 100 rows, columns: ID, Name, Age, Salary, Department
# - Introduce 10% missing values randomly
# - Clean the data:
#   1. Fill Age with median
#   2. Fill Salary with mean by Department
#   3. Fill Name with 'Unknown'
#   4. Drop rows where all values are NaN
```

Exercise 4.2: GroupBy Analysis

```
# Using sales data:
# 1. Total sales by product
# 2. Average sales by region
# 3. Top 3 products by sales per region
# 4. Monthly trend for each product
# 5. Year-over-year growth
```

Exercise 4.3: Merge DataFrames

```
# Create 3 DataFrames:
# - Orders (order_id, customer_id, product_id, quantity)
# - Customers (customer_id, name, city)
# - Products (product_id, product_name, price)
# Merge to create complete order details with:
# customer name, product name, total amount
```

Exercise 4.4: Reshape Data

```
# Given wide format exam scores:
# Name, Math, English, Physics, Chemistry
# Convert to long format:
# Name, Subject, Score
# Then pivot to show average by Subject
```

Summary | Tóm tắt

Missing Data

Function	Description
<code>isnull()</code>	Detect missing
<code>dropna()</code>	Remove missing
<code>fillna()</code>	Fill missing
<code>interpolate()</code>	Interpolate

GroupBy

Function	Description
<code>groupby()</code>	Create groups
<code>agg()</code>	Multiple aggregations
<code>transform()</code>	Apply, keep shape
<code>filter()</code>	Filter groups

Merge/Join

Function	Description
<code>concat()</code>	Stack DataFrames
<code>merge()</code>	SQL-like join
<code>join()</code>	Join on index

Next Lesson | Bài tiếp theo

Lesson 5: Matplotlib

- Basic plots
- Customization

- Subplots
- Saving figures